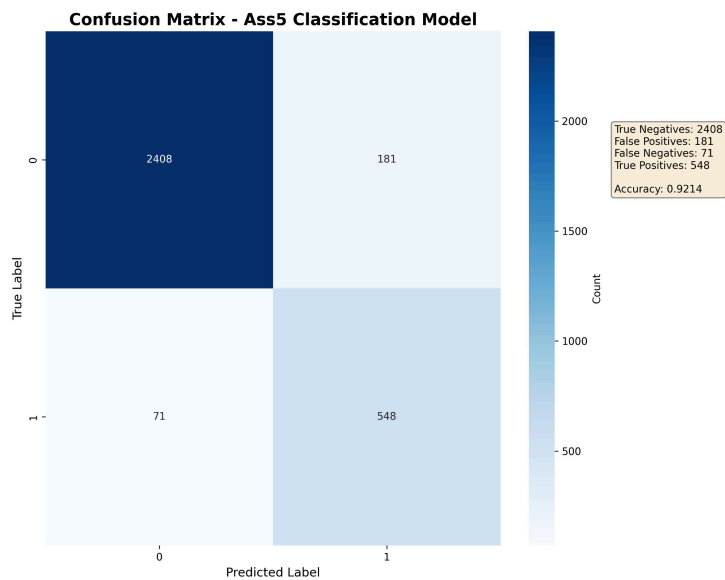


6758003756 Kijputhip Oonsonk



Results:

Data preparation

Data preparation. I loaded the ass5 dataset, normalized column names, cleaned categorical text, and handled missing values by imputing numeric fields with the median and categorical fields with the mode. The target ProdTaken was kept as a binary label. I then applied a stratified holdout split using train_test_split with test_size=0.2, producing an 80% training set and 20% test set while preserving class distribution. During model training, Keras used validation split=0.2 on the training portion, so the effective split is approximately 64% train, 16% validation, 20% test. Categorical fields were one-hot encoded, and numerical features were standardized using a StandardScaler fitted on the training data and reused for inference to avoid leakage.

Analysis

The dataset combines demographic variables (e.g., age, marital status, income), engagement behavior (e.g., duration of pitch, number of follow-ups, pitch satisfaction), and product context (e.g., product pitched, preferred property star), which together provide strong signals for conversion prediction (ProdTaken). The target is imbalanced toward non-purchase cases, so relying only on accuracy could hide weak minority-class performance; this is why class weighting and recall/AUC monitoring are appropriate. Behavioral features are likely the most predictive because they directly reflect customer intent and interaction quality, while demographic features provide supporting segmentation context. Overall, the data is suitable for supervised binary classification, with enough mixed feature types to justify encoding + scaling before neural-network training.

Feature extraction

The prediction target is whether a customer purchases the product (ProdTaken=1) or not (ProdTaken=0). To predict this, I used customer profile and interaction features (age, occupation, city tier, income, product pitched, contact type, follow-ups, satisfaction score, travel history, and related attributes). Nominal categorical inputs were transformed with one-hot

encoding so the model can learn separate effects for each category without implying false ordering. Numeric inputs were standardized so dense layers train stably and no high-scale variable dominates gradients. I also reindexed inference features to the exact training feature list so the model always receives the same columns in the same order, preserving feature meaning across training and deployment.

Building the model. I built a feed-forward neural network with three hidden layers (128 → 64 → 32 units), ReLU activations, and dropout for regularization, followed by a sigmoid output for binary classification. The model was optimized with Adam and trained using binary cross-entropy loss, plus AUC and accuracy metrics. Early stopping and learning-rate reduction were used to stabilize training and prevent overfitting, and class weights addressed the label imbalance.

Evaluation results. On the held-out test split, accuracy is about 0.9214, and recall for the positive class is $548/(548+71) \approx 0.8853$. The confusion matrix indicates strong true-negative performance with a moderate number of false positives and a smaller number of false negatives, aligning with the goal of high overall accuracy while maintaining good recall for positive cases.